

# Quotas gratuits sur les épreuves et examens nationaux — guide mobile

Ce que voit un utilisateur **sans abonnement** : 5 ressources académiques distinctes, et Ketsia sur une seule d'entre elles. Toutes les réponses ci-dessous sont issues d'**appels réels** sur l'environnement de développement.

Référence : issue #245. Complète le [guide des abonnements](#).

## 1. La règle

| ressource                    | sans abonnement                    | avec abonnement |
|------------------------------|------------------------------------|-----------------|
| épreuves + examens nationaux | 5 distinctes par mois, pool commun | illimité        |
| Ketsia                       | 1 ressource par mois               | illimité        |
| concours                     | aucun accès gratuit (#244)         | illimité        |

Les plafonds sont **réglables depuis le back-office** et peuvent changer sans déploiement. Lisez-les dans `quota.limit`, ne les codez jamais en dur.

Trois propriétés à retenir, chacune visible dans l'interface :

- **Le quota porte sur des ressources distinctes, pas sur des accès.** Ouvrir

trois fois la même épreuve dans le mois consomme 1 sur 5. Rouvrir une ressource déjà décomptée reste possible **même quota épuisé**.

- **Le pool est commun.** 3 épreuves + 2 examens nationaux épuisent les 5.
- **Il se remet à zéro chaque mois**, le 1er à 00:00 UTC. La date exacte est

renvoyée par l'API — ne la recalculez pas côté client.

⚠ Conséquence à ne pas manquer : `deja_consultee` vaut « déjà consultée **ce mois-ci** ». Une ressource lue en septembre redevient payante en unité au 1er octobre. N'entretenez pas de cache local de ce drapeau d'un mois sur l'autre.

Ketsia a son **propre** compteur : épuiser les 5 ressources ne consomme pas le lancement Ketsia, et inversement.

### ⚠ Le verrou est éteint aujourd'hui

Comme pour les concours, le serveur démarre avec `ABONNEMENTS_VERROU_ACTIF=false` : le refus est **journalisé mais pas appliqué**. `verrouille`, `deja_consultee` et `mes-droits` renvoient malgré tout la vérité. **Développez contre ces valeurs** ; le jour de la bascule, rien ne changera côté application.

## 2. Les droits — `GET /abonnements/mes-droits?country=benin`

```
{
  "verrou_actif": false,
  "droits": {
    "CONCOURS_DOWNLOAD": { "allowed": false, "reason": "SUBSCRIPTION_REQUIRED" },
    "EPREUVE_VIEW":      { "allowed": true,  "reason": "FREE_QUOTA", "quota": { "used": 0, "limit": 5 } },
    "EXAMEN_NAT_VIEW":   { "allowed": true,  "reason": "FREE_QUOTA", "quota": { "used": 0, "limit": 5 } },
    "KETSIA_AI":         { "allowed": true,  "reason": "FREE_QUOTA", "quota": { "used": 0, "limit": 1 } },
    "AI_STATS":          { "allowed": false, "reason": "SUBSCRIPTION_REQUIRED" }
  }
}
```

Les décisions **divergent** désormais d'une fonctionnalité à l'autre : lisez celle qui vous concerne, ne déduisez pas les autres.

| reason                | sens                                                     |
|-----------------------|----------------------------------------------------------|
| SUBSCRIBED            | abonnement actif — aucun quota                           |
| ADMIN                 | compte administrateur — jamais bloqué                    |
| FREE_QUOTA            | autorisé sur le quota gratuit ; quota dit où l'on en est |
| QUOTA_EXCEEDED        | quota épuisé — proposer l'abonnement                     |
| SUBSCRIPTION_REQUIRED | aucun accès gratuit sur cette fonctionnalité             |

EPREUVE\_VIEW et EXAMEN\_NAT\_VIEW partagent le même compteur : leurs quota sont toujours identiques. C'est le pool commun vu sous deux angles.

Le compteur seul — GET /abonnements/mes-quotas?country=benin

```
{
  "RESOURCE_VIEW": { "used": 1, "limit": 5, "est_actif": true, "periode_reset": "MENSUEL", "reinitialisation": "2026-10-01T00:00:00.000Z" },
  "KETSIA_AI": { "used": 0, "limit": 1, "est_actif": true, "periode_reset": "MENSUEL", "reinitialisation": "2026-10-01T00:00:00.000Z" }
}
```

| champ            | usage                                                                |
|------------------|----------------------------------------------------------------------|
| used / limit     | « il vous reste 4 ressources gratuites »                             |
| reinitialisation | date de remise à zéro — « vos ressources reviennent le 1er octobre » |
| periode_reset    | MENSUEL ou AVIE ; ne supposez pas la valeur                          |
| est_actif        | false = quota désactivé, la fonctionnalité est libre                 |

Affichez la date de remise à zéro dans le message de blocage. « Revenez le 1er octobre, ou abonnez-vous » se refuse mieux qu'un simple « quota épuisé ».

Quand est\_actif vaut false, l'administration a levé le quota : mes-droits renvoie alors FREE\_QUOTA sans objet quota. Traitez son absence comme « illimité », pas comme zéro.

3. Les listes portent deux drapeaux

GET /epreuves et GET /epreuves/:id renvoient sur chaque élément :

```
{ "id": 6, "titre": "BIostatistique – normal", "verrouille": false, "deja_consultee": false }
```

| champ          | sens                                                        |
|----------------|-------------------------------------------------------------|
| verrouille     | ouvrir cette ressource sera refusé une fois le verrou actif |
| deja_consultee | elle est déjà décomptée : l'ouvrir ne coûtera rien          |

deja\_consultee n'est pas décoratif. Sans lui, l'application ferait croire qu'ouvrir une ressource déjà lue consomme une nouvelle unité — et l'utilisateur s'interdirait d'y revenir. Concrètement, avec un quota épuisé :

```
epreuve 6   verrouille=true   deja_consultee=false   → cadenas
epreuve 10  verrouille=false  deja_consultee=true    → ouvrable, badge « déjà consultée »
epreuve 11  verrouille=true   deja_consultee=false   → cadenas
```

Une ressource deja\_consultee a toujours verrouille: false, quota épuisé ou non. Les deux drapeaux sont calculés en une requête par appel HTTP.

## 4. Ouvrir une ressource

### Épreuves — GET /epreuves/:id/telechargement?country=benin

Le quota est consommé **avant** que les octets partent. Réponse en cas de refus :

```
{
  "statusCode": 403,
  "error": "QUOTA_EXCEEDED",
  "message": "Vous avez consulté vos 5 ressources gratuites. Un abonnement est requis pour continuer.",
  "feature": "EPREUVE_VIEW",
  "quota": { "used": 5, "limit": 5 }
}
```

### Examens nationaux — GET /files/examens\_nationaux/:uuid/file/download-url?country=benin

Rien ne change dans votre appel : c'est la route que vous utilisez déjà. Elle consomme maintenant le même quota et renvoie le même refus, avec "feature": "EXAMEN\_NAT\_VIEW".

Il n'y a volontairement pas de nouvelle route /examens-nationaux/:id/telechargement : verrouiller un chemin en laissant l'autre ouvert n'aurait rien verrouillé du tout.

Traitez le 403 **même si** vous respectez verrouille : le quota peut s'épuiser entre l'affichage de la liste et le tap, ou depuis un autre appareil.

## 5. Ketsia — POST /abonnements/quota/ketsia

À appeler **avant** d'ouvrir l'assistante.

```
{ "resource_type": "epreuve", "resource_id": 6, "pays": "benin" }
```

resource\_type vaut epreuve ou examen\_national.

### Réponse — 201

```
{ "allowed": true, "reason": "FREE_QUOTA", "quota": { "used": 1, "limit": 1 } }
```

### Refus — 403

```
{
  "statusCode": 403,
  "error": "QUOTA_EXCEEDED",
  "message": "Vous avez utilisé Ketsia sur votre ressource gratuite. Un abonnement est requis pour continuer.",
  "feature": "KETSIA_AI",
  "quota": { "used": 1, "limit": 1 }
}
```

Revenir sur la ressource **déjà décomptée** est toujours autorisé et ne consomme rien — l'utilisateur peut poursuivre sa conversation autant qu'il veut.

Cet appel est un **confort d'interface**, pas la sécurité : il vous évite d'ouvrir un écran qui échouera. Le contrôle qui fait foi est celui que Kessiah effectue de serveur à serveur. Ne construisez donc pas de logique qui suppose qu'un appel omis donne un accès gratuit.

## 6. Ce que l'écran doit faire

**Afficher le reste, pas le consommé.** « Il vous reste 2 ressources gratuites » se lit mieux que « 3/5 utilisées ». Les deux chiffres sont dans quota.

**Distinguer cadenas et « déjà consultée ».** Deux états visuels différents : un cadenas invite à s'abonner, un badge « déjà consultée » rassure sur la gratuité.

**Prévenir avant la dernière unité.** À `used === limit - 1`, un message au moment d'ouvrir — « c'est votre dernière ressource gratuite » — évite le sentiment d'avoir été piégé.

**Ne pas décompter côté client.** Le compteur vit sur le serveur et vaut pour tous les appareils. Rafraîchissez `mes-quotas` après chaque ouverture plutôt que d'incrémenter localement.

**Router le 403 vers l'abonnement.** `error === "QUOTA_EXCEEDED"` et `error === "SUBSCRIPTION_REQUIRED"` mènent au même écran ; seul le message diffère.

## 7. Exemple Flutter

```
class QuotasApi {
  QuotasApi(this._client, {required this.baseUrl, required this.token, required this.pays});
  final http.Client _client;
  final String baseUrl, token, pays;

  Map<String, String> get _entetes =>
    {'Authorization': 'Bearer $token', 'Content-Type': 'application/json'};

  Future<Map<String, Quota>> mesQuotas() async {
    final r = await _client.get(Uri.parse('$baseUrl/abonnements/mes-quotas?country=$pays'), headers: _entetes);
    final d = jsonDecode(r.body) as Map<String, dynamic>;
    return d.map((k, v) => MapEntry(k, Quota(used: v['used'], limit: v['limit'])));
  }

  /// À appeler AVANT d'ouvrir l'assistante. Confort d'interface : Kessiah
  /// contrôle de son côté, ne comptez pas sur cet appel pour la sécurité.
  Future<bool> ouvrirKetsia(String type, int id) async {
    final r = await _client.post(
      Uri.parse('$baseUrl/abonnements/quota/ketsia'),
      headers: _entetes,
      body: jsonEncode({'resource_type': type, 'resource_id': id, 'pays': pays}),
    );
    if (r.statusCode == 403) return false;
    return r.statusCode ~/ 100 == 2;
  }
}

class Quota {
  const Quota({required this.used, required this.limit, this.reinitialisation});
  final int used, limit;
  /// Date de remise à zéro renvoyée par le serveur – ne la recalculiez pas :
  /// la période est réglable depuis le back-office (mensuelle ou à vie).
  final DateTime? reinitialisation;
  int get restant => (limit - used).clamp(0, limit);
  bool get epuise => used >= limit;
  bool get derniere => restant == 1; // pour prévenir avant le dernier usage
}
```

### Ouvrir une ressource

```
Future<void> ouvrirEpreuve(Epreuve e) async {
  // `deja_consultee` prime : la ressource est acquise, aucun avertissement.
  if (e.verrouille && !e.dejaConsultee) return ouvrirEcranAbonnement();

  if (!e.dejaConsultee && quotas['RESOURCE_VIEW']!.derniere) {
    final continuer = await confirmer('C'est votre dernière ressource gratuite. Continuer ?');
    if (!continuer) return;
  }

  final r = await client.get(
    Uri.parse('$baseUrl/epreuves/${e.id}/telechargement?country=$pays'),
    headers: _entetes,
  );

  if (r.statusCode == 403) {
    final corps = jsonDecode(r.body);
    // QUOTA_EXCEEDED et SUBSCRIPTION_REQUIRED mènent au même écran.
    return ouvrirEcranAbonnement(quota: corps['quota']);
  }

  await rafraichirQuotas(); // le compteur vit sur le serveur
}
```

## 8. Rappels

---

**Le pool est commun** aux épreuves et aux examens nationaux : `EPREUVE_VIEW` et `EXAMEN_NAT_VIEW` renvoient toujours le même `quota` .

**Ketsia a son propre compteur**, indépendant des 5 ressources.

**Une ressource déjà consultée reste ouverte**, quota épuisé ou non.

**Le quota se remet à zéro chaque mois** — affichez `reinitialisation` , ne la recalculez pas.

**Les plafonds sont administrables** et peuvent changer sans déploiement : lisez toujours `limit` , ne codez jamais 5 ni 1 en dur.

Un `quota` absent avec `FREE_QUOTA` signifie « illimité », pas zéro.

**Aucun changement d'appel pour les examens nationaux** — `download-url` reste la route, elle est simplement soumise au quota.

**Le verrou est éteint aujourd'hui**, mais `verrouille` , `deja_consultee` et `mes-droits` disent déjà la vérité : construisez l'interface contre eux.